Kaitlyn Freeley, Jazmyn Harris, Natasha Nicholas

Professor Rose Sloan

CS5100: Intro to AI

April 15, 2026

Final Project Report: Pacman Q-Learning

Introduction

For our final project, we decided to look into how an AI might approach playing a free moving game like Pacman. We have seen in class how AIs approach similarly formatted games using reinforcement learning. Reinforcement learning uses a reward system to motivate the AI to prioritize certain actions over others. To prioritize the reinforcement learning aspect of the project rather than focusing on accurate gameplay, we chose to exclude classic game features such as multiple lives, pellet powerups, and fruits that provide extra points. For our model, we considered using a single-agent versus a multi-agent reinforcement learning. In single agent reinforcement learning (SARL), the agent assumes nothing in its environment is changing, whereas in multi-agent reinforcement learning (MARL), each agent assumes the environment is not stationary because the other agents are moving within it as well (Canese et al., 2021). Because we are including ghosts that move independently of the Pacman sprite, we decided to approach our project as a MARL problem. We use the Q-learning method of reinforcement learning to train our AI to find optimal solutions from its current position in the game. Even if the AI does not choose to go down these paths, it can still discover unexplored states that may be more optimal than following the current policy.

Method

To solve our problem, we looked into coding a Q-learning function. We decided to use Q-learning because we wanted our AI to try optimal paths, regardless of whether Pacman goes down this path or not. By doing this, we can explore actions Pacman can take in various scenarios, such as being approached by a ghost, being equidistant from multiple pellets, and what it might do when it is close to a pellet but being approached by a ghost. We also had various rewards and penalties to determine what Pacman should prioritize when approached with multiple scenarios. Our win state is a high positive reward when there are no pellets in the maze, while the lose state is a high negative reward when a ghost catches Pacman. These high rewards will overpower the other rewards we use, so that the AI will prioritize winning the game and avoiding getting caught by a ghost over everything else. When running Q-learning, we first train the AI using 15000 episodes, thus giving it ample time to learn the different methods to use. During this training, we print the overall rewards to determine the AI's success rate throughout the training.

Results

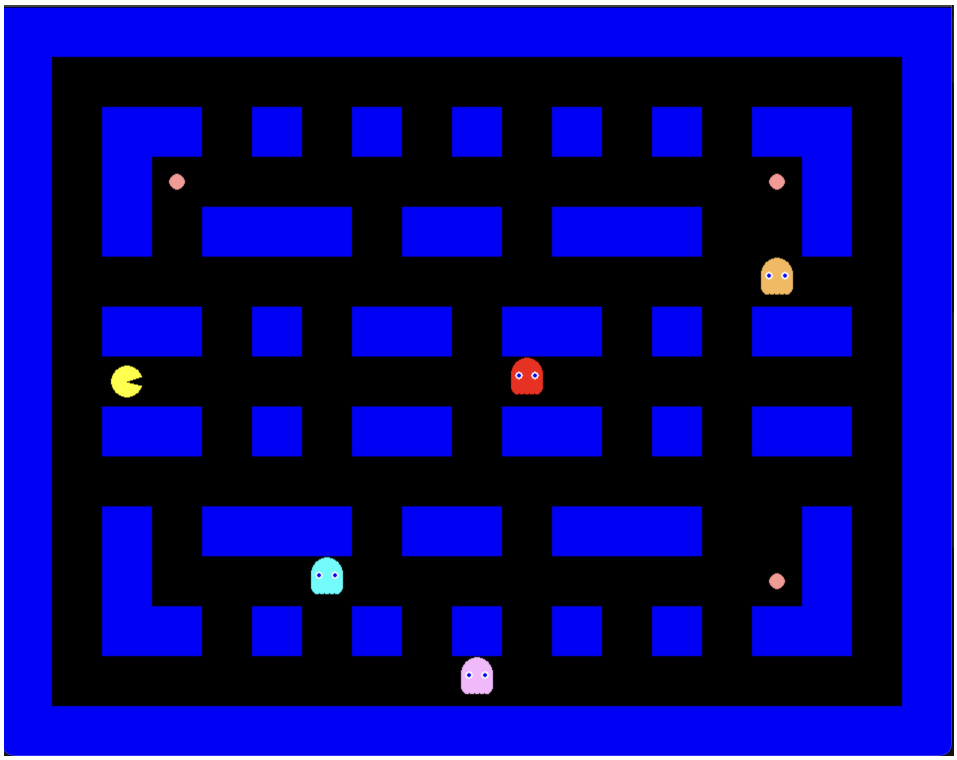
In our project, we each tested various different combinations of parameters of Q-learning and the reward values of the PacMan environment. We did this to find the best combination of values that would result in a high win percent. All of our final models were trained with the Q_learning parameters: num_episodes=15000, gamma=0.85, epsilon=1.0, decay_rate=0.999, and alpha=0.1. These were the values that we all found to produce the best values for this problem. We wanted to prioritize future rewards without compromising immediate rewards as well, and found that setting the gamma to 0.85 created a good balance (Bousaid et al. 2025). The rest of

the values were tweaked for improvement but found little improvement from personal assignment 2. Our update policy adhered to Equation 1.

Equation 1: $Q(s,a) <- Q(s,a) + \alpha(r + \gamma * max_{a'} Q(s',a') - Q(s,a)$

We also made sure to standardize the maze that we used in order to make an accurate comparison. Therefore, we compared our 15,000 episode models on the Grid City Maze (Figure 1).

Figure 1: Grid City Maze



Model 1: Kaitlyn's Model

In Kaitlyn's code, we included various rewards to control Pacman's movement. The rewards appear as follows:

```
  'pellet': 100,
'ghost': -500,
'empty': -0.1,
'closer': 3,
'further': -2,
'win': 2000,
'oob': -5
```

In these rewards, we see how the AI is rewarded for collecting a pellet, getting closer to a pellet, and winning the game (ie. collecting all the pellets). It is penalized for moving further from a pellet, being in an empty space, going out of bounds, or running into a ghost (ie. losing the game). We chose to make the win reward the largest one, as we want the AI to prioritize winning the game. We did not make the ghost penalty as large, however, because we noted that Pacman would be too afraid to collect a pellet if a ghost was near it. Because the pellet reward is significantly lower than the ghost reward, the AI focuses more on escaping the ghosts than collecting the pellets. By decreasing the penalty value but still having it larger than the pellet reward, we ensure that Pacman remains relatively afraid of ghosts while still having the confidence to collect a nearby pellet.

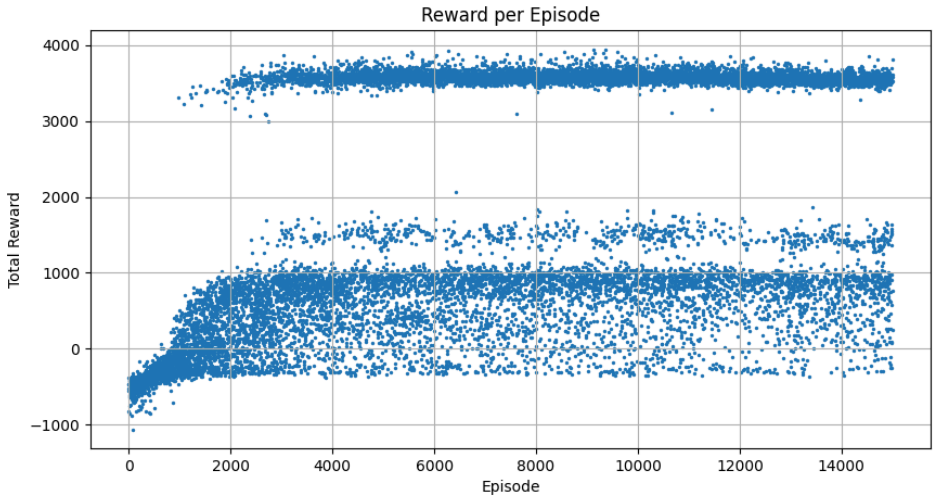
To further incentivize Pacman to collect more pellets, we added additional rewards and penalties regarding proximity to the pellets. If Pacman moved closer to a given pellet, it would receive a reward of 3, and if it moved further away, it received a penalty of -2. We also included a small penalty on any movement to a square that did not contain a pellet, to dissuade the AI to fall back on empty spots unless necessary. Finally, we have a penalty for going out of bounds to ensure Pacman remains within the walls of the maze.

When the model was run with 15,000 episodes, the results are as follows:

```
==================================
Training complete  (15000 episodes)
Total wins         : 6630  (44.2%)
Last 100 win%      : 53.0%
Last 500 win%      : 55.0%
Avg reward (last 500): 2326.5
==================================
Evaluating (1000 softmax episodes) …
Softmax win rate : 419/1000  (41.9%)
Avg reward       : 1912.5
```

We can see that in the overall training, the agent won 44.2% of the time. In the last 500 episodes, it won 55.0% of the time, while in the last 100 episodes, it won 53.0% of the time, showing how the model continued to be consistent. In Figure 2, we see how the agent begins to earn more high positive rewards around 1000 episodes, showing how the agent begins to win very early on. As the episodes progress, the high rewards remain consistent, but the negative rewards still remain, though spread out. This shows that while the model learns to win consistently, it still tends to lose occasionally, as seen in the win percentage data provided above.

Figure 2: Rewards per Episode Training Graph on Model 1



Model 2: Natasha's Model with Reward Tweaking

In Natasha's environment, the rewards appear as follows:

```
'pellet': 100,
'ghost': -500,
'empty': -1,
'closer': 1,
'further': -1,
'win': 2000,
'oob': -5
```

For comparison, we can see that the rewards and penalties for the pellets, ghost collision, winning state, and out of bounds all remained the same. However, in this model, we altered how the penalties and rewards for the proximity to the pellets were applied. Firstly, we wanted to see if increasing the empty penalty to a full point would impact Pacman's behavior. This is to slightly emphasize the importance of making meaningful moves while still allowing the agent to roam freely. We noticed that Pacman would occasionally loiter on the board, likely to increase the total reward gained by moving closer to a pellet without actually collecting it. To avoid this behavior, we attempted to equalize the closer and further reward, so that the agent is still pushed to move closer to pellets, but cannot use a loophole to increase the total reward.

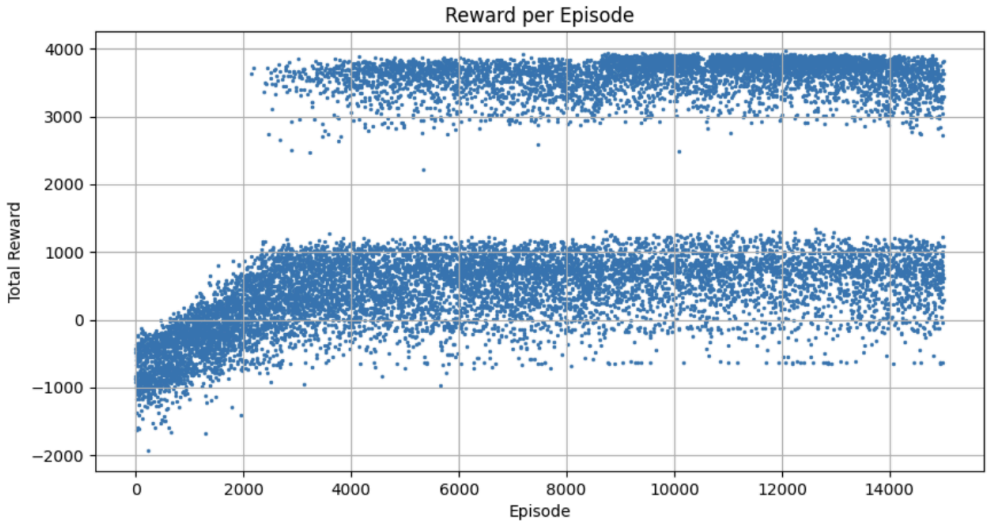
When the model was run with 15,000 episodes, the results were as follows:

```
==================================
Training complete  (15000 episodes)
Total wins         : 5455  (36.4%)
Last 100 win%      : 37.0%
Last 500 win%      : 47.0%
Avg reward (last 500): 1964.4
==================================
Evaluating (1000 softmax episodes) …
Softmax win rate : 379/1000  (37.9%)
Avg reward       : 1619.6
```

In the training, we see that the agent won 36.4% of the time. In the last 500 episodes, it won 47.0% of the time, while in the last 100 episodes it won 37.0% of the time, thus showing how the model did not improve over time. In Figure 3, we see how the agent learns over time.

After about 2000 episodes, the agent begins to achieve higher positive rewards, thus winning more often. However, we can see that as the high positive rewards increase, there are still a large number of negative rewards, showing that despite the agent learning how to win, it tends to lose very often as well.

Figure 3: Rewards per Episode Training Graph on Model 2



Model 3: Jazmyn's Model with Extra Rewards

In Jazmyn's code, there were extra reward states to incentivise and disincentivise some of the behavior that we saw prominent in the two previous models. The first negative behavior that we saw occur in the previous models was what we coined as "loitering". We defined loitering as two different scenarios and set a negative reward for this behavior. Loitering can occur when the agent is seen in the same 2 or less boxes (2x2 grid around the player) over a period of 6 steps. The second way that it can occur is by frequenting the same positions over a period of 8 steps. This significantly reduced the agent remaining in the same place and it moved more confidently

towards pellets. The next negative behavior that we noticed is that the agent lacked fear when it was near a ghost. This was mitigated by introducing positive and negative rewards for moving further or closer to the nearest ghost. Lastly, we added a high negative reward if the agent was in the “danger zone” of a ghost (within 3 manhattan distance steps). The final rewards were balanced as follows:

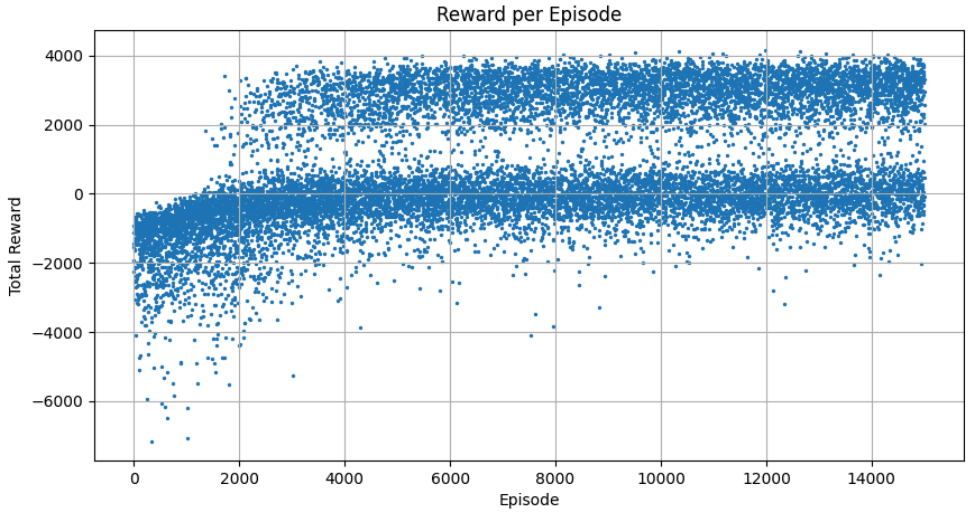
```
'pellet': 100,
'ghost': -500,
'empty': -0.1,
'closer_to_pellet': 10,
'further_from_pellet': -2,
'close_to_ghost': -5,
'further_from_ghost': 2,
'danger_zone': -20,
'loitering': -6,
'win': 2000,
'oob': -5
```

When this model was run with 15,000 episodes we garnered the following results.

```
================================
Training complete  (15000 episodes)
Total wins         : 5769  (38.5%)
Last 100 win%      : 55.0%
Last 500 win%      : 55.8%
Avg reward (last 500): 1678.8
================================
Evaluating (1000 softmax episodes) …
Softmax win rate : 554/1000  (55.400000000000006%)
Avg reward       : 1619.6
```

From the training, we see that the agent won 38.5% of the time. In the last 100 episodes, it won 55.0% of the time. Similarly, in the last 500 episodes it won 55.8% of the time. In Figure 4, we can see the evolution of the agent as it learns. Around episode 2000 of the graph, we see that the agent begins to achieve high positive rewards. This means that it is learning to win more. As the episodes progress, we no longer see points in the extreme negative region. Additionally, the points in the winning region (2000+ reward) become more dense and begin to match the density of the losing region (<1000 reward).

Figure 4: Rewards per Episode Training Graph on Model 3

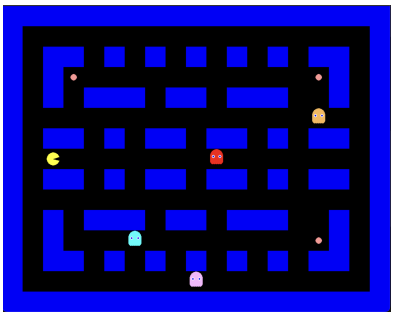
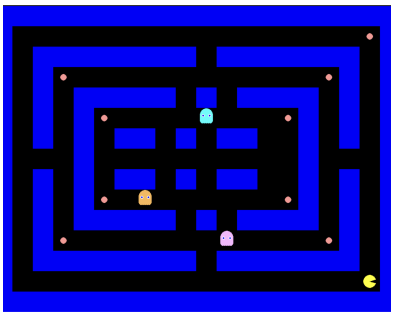
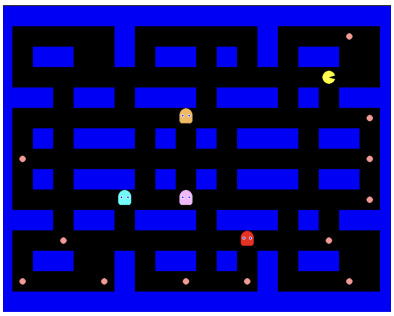


When comparing the three models, we can see how the rewards vary. In Natasha's model, the agent learns to win around 2000 episodes; however, the model seems to win and lose equally, suggesting that the different rewards are not as effective. This is further supported by the softmax win rate of 37.9%, which is the lowest win rate of the three models. In Kaitlyn's model, we see how the agent also learns to win around 2000 episodes, but is more consistent in winning the games, with a softmax win rate of 41.9%. This shows that the more drastic pellet proximity rewards and penalties were more effective in influencing the agent to make meaningful decisions. Finally, in Jazmyn's model, we see that the model again begins to win around 2000 episodes. However, in this model, the agent seems to win more frequently compared to the other two models, as supported with the softmax win rate of 55.4%, the highest win rate of the three. This win rate shows how the extra rewards and penalties, such as loitering, proximity to ghost, and danger zone, proved to be the most effective at training the AI. This is because it provides

more useful information for the agent, thus preventing it from finding loopholes when gaining rewards and instead making meaningful decisions.

Lastly, we examined how this model performs on different maps. The agent's performance obviously varied significantly across the three maze designs that we tested. This highlights how the environment structure directly affects the reinforcement learning outcome. All of the previous results have been tested on Grid City (Figure 1). However, we wondered if this maze was the most optimal for our reinforcement learning models. To understand, we tested the model on two other mazes of different structures (Figure 5).

Figure 5: Jazmyn's model comparison on different mazes

Grid City	**Rings**	**Catacombs**
		
==========	==========	==========
Training complete (5000 episodes)	Training complete (5000 episodes)	Training complete (5000 episodes)
Total wins : 728 (14.6%)	Total wins : 21 (0.4%)	Total wins : 970 (19.4%)
Last 100 win% : 31.0%	Last 100 win% : 0.0%	Last 100 win% : 33.0%
Last 500 win% : 29.4%	Last 500 win% : 0.8%	Last 500 win% : 35.8%
Avg reward (last 500): 608.1	Avg reward (last 500): -778.6	Avg reward (last 500): 920.2
==========	==========	==========
Evaluating (1000 softmax episodes)	Evaluating (1000 softmax episodes)	Evaluating (1000 softmax episodes)
…	…	…
Softmax win rate : 274/1000 (27.4%)	Softmax win rate : 7/1000 (0.7%)	Softmax win rate : 330/1000 (33.0%)
Avg reward : 285.4	Avg reward : -891.2	Avg reward : 577.2

Grid City produced moderate results, with a final evaluation win rate of 27.4% and stable positive rewards. Its structure features long, straight corridors and frequent intersections, making it easy for the agent to learn consistent movement patterns and move towards pellets. But it also restricted deeper strategic learning. Once the basic policies were learned, the improvement plateaued. In contrast, the Rings maze was dramatically worse with a near-zero win rate (0.7%) and highly negative reward. This told us that the agent does not perform well with a layout that has sparse junctions and concentric loops as it forces the agent to commit to long paths with limited escape options. The final maze, Catacombs, was the best performing. It had a 33.0% win rate on the softmax evaluation and had the highest average reward. Although it had more dead ends, the frequent junctions, narrow corridors and pockets provided richer learning signals.

Conclusion

Based on the results above, we can see that Jazmyn's reward system proved to be the most effective, as it provided more useful information for the agent. It incentivized the agent to make meaningful decisions to avoid being penalized, and dissuaded it from utilizing loopholes that may have been seen in Kaitlyn and Natasha's models. The final softmax win rate for Jazmyn's model was 55.4%, showing that the agent won a little over half of the episodes.

Overall, the reinforcement learning approach using Q-learning demonstrated clear strengths in its simplicity, interpretability, and ability to learn effective policies in structured environments, as seen in the strong performance on the Catacombs and moderate success on the Grid City maze. The tabular Q-learning framework allowed for straightforward implementation and debugging, and it performed well when the environment provided frequent, consistent reward signals and repeatable spatial patterns. However, the approach also revealed key

weaknesses, particularly its sensitivity to environment structure and its difficulty handling sparse rewards and long-term dependencies, which was evident in the poor performance on the Rings maze. Compared to more advanced methods such as Deep Q-Networks (DQN), which approximate value functions and generalize across larger state spaces, tabular Q-learning struggles with scalability and requires extensive exploration to converge in complex layouts. Additionally, the lack of memory or planning capability limits its effectiveness in environments that require backtracking or long-horizon decision making.

One promising direction for improvement is the use of enhanced update strategies such as Dynamic Q-Learning (DQL), proposed by Mariano and Morales, which modifies the Q-value update rule to accelerate convergence and improve stability in environments with delayed rewards. Incorporating such methods, along with techniques like function approximation or prioritized experience replay, could significantly improve learning efficiency and robustness across more complex maze configurations.

Another idea we could implement is adding hyperparameters in our Q-learning function, as suggested by Bousaid, Kaabi, Dhraief, and Drira. They discuss how to fine tune the Q-learning algorithm by adjusting them based on linear functions, exponential functions, hybrid functions, and grid-search algorithms (Bousaid et al., 2025). Comparing these different Q-learning models as well may show how the agent may learn better or worse based on the various functions, and we may see more improvements on our results.

Bibliography

Bousaid, M., Kaabi, S., Dhraief, A. *et al.* (2025). Hyperparameters Tuning in Adaptive Q-Learning and DQN-Based Routing Protocols for FANETs: A Comparative Study. *SN COMPUT. SCI.* 6, 459. https://doi.org/10.1007/s42979-025-03994-3

Canese, L., Cardarilli, G. C., Di Nunzio, L., Fazzolari, R., Giardino, D., Re, M., & Spanò, S. (2021). Multi-Agent Reinforcement Learning: A Review of Challenges and Applications. *Applied Sciences*, *11*(11), 4948. https://doi.org/10.3390/app11114948

Mariano, C.E., Morales, E.F. (2001). DQL: A New Updating Strategy for Reinforcement Learning Based on Q-Learning. In: De Raedt, L., Flach, P. (eds) Machine Learning: ECML 2001. ECML 2001. Lecture Notes in Computer Science(), vol 2167. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-44795-4_28